M5 — Mel front-end parity and the neural tangent kernel

jax-experiments writeup · M5

2026-10-16

Contents

N7 — The empirical neural tangent kernel **2**
- Question . . . 2
- The idea . . . 2
- The math . . . 2
- What we measure and why . . . 3
- What I built . . . 4
- Results . . . 4
- What surprised me . . . 7
- Open threads . . . 7

O7 — Mel front-end parity **8**

Status: N7 done; O7 (mel front-end parity) not started ·
Hardware: CPU, 4 cores, x86_64. No GPU was used ·
Data: `data/n7-kernel.json`, `data/n7-train.json`, `data/n7-alpha.json`, `data/n7-alpha-gelu.json`, `data/n7-converged.json`, `data/n7-floor.json` ·
Reproduce: `uv run python experiments/n7_ntk/run.py kernel train alpha alpha-gelu converged floor plot` (about 25 minutes; needs `../ml-ladder` for B1's moons generator; MNIST downloads once into `data/`)

N7 — The empirical neural tangent kernel

Question

How well does the infinite-width neural tangent kernel (NTK) predict what a finite network actually does, as width grows? And when does feature learning break the agreement?

The idea

Training is linear in function space. A network $f(x; w)$ trained by gradient descent on squared loss moves its outputs as

$$\dot{f} = -\eta\,\Theta_w(f - y)/n, \qquad \Theta_w(x, x') = \langle \nabla_w f(x), \nabla_w f(x') \rangle.$$

At infinite width the kernel freezes (Jacot et al., 2018). In the NTK parameterisation, Θ_w converges to a deterministic kernel Θ_∞ at initialisation, and stays there during training. The network then *is* kernel regression with Θ_∞, a prediction made without training a network at all.

Finite width breaks this two ways.

- At initialisation, Θ_{w_0} fluctuates around Θ_∞, with a relative error of order $1/\sqrt{\text{width}}$.
- During training, Θ_w moves. That movement is feature learning, and the kernel description fails exactly when it is large.

A knob separates the two (Chizat, Oyallon & Bach, 2019). Train the centred, scaled model $g = \alpha\,(f(w) - f(w_0))$ on $\frac{1}{2\alpha^2}$ times the squared loss:

- Its function-space dynamics at $t = 0$ are the same for every α.
- To move g by $O(1)$, the weights move by $O(1/\alpha)$.
- Large α is therefore "lazy": the kernel should stay put, and g should track its own linearisation with an error of $O(1/\alpha)$. Small α forces the weights, and the features, to move.

The math

Network. Two hidden ReLU layers and a scalar output, NTK parameterisation, $\sigma_w^2 = 2$, $\sigma_b^2 = 0.1$:

$$h_1 = \sigma_w W_1 x/\sqrt{d} + \sigma_b b_1, \quad h_l = \sigma_w W_l\,\phi(h_{l-1})/\sqrt{n} + \sigma_b b_l, \quad f = \sigma_w w^\top \phi(h_L)/\sqrt{n} + \sigma_b b.$$

Analytic NTK (Jacot et al.; the arc-cosine kernels of Cho & Saul, 2009). For (u, v) Gaussian with covariance $\begin{pmatrix} k_{11} & k_{12} \\ k_{12} & k_{22} \end{pmatrix}$ and $\theta = \arccos\left(k_{12}/\sqrt{k_{11}k_{22}}\right)$:

$$\mathbb{E}[\phi(u)\phi(v)] = \frac{\sqrt{k_{11}k_{22}}}{2\pi}(\sin\theta + (\pi-\theta)\cos\theta), \qquad \mathbb{E}[\phi'(u)\phi'(v)] = \frac{\pi-\theta}{2\pi}.$$

The recursion runs through the layers:

$$\Sigma^l = \sigma_w^2\,\mathbb{E}[\phi\phi](\Sigma^{l-1}) + \sigma_b^2, \qquad \Theta^l = \Theta^{l-1}\,\sigma_w^2\,\mathbb{E}[\phi'\phi'](\Sigma^{l-1}) + \Sigma^l.$$

Empirical NTK, without the Jacobian. For a dense layer, $\partial f/\partial W_l = \delta_l\, a_{l-1}^\top$, an outer product, where $\delta_l = \partial f/\partial h_l$ comes from one vector-Jacobian product. So

$$\Theta_w = \sum_l (\Delta_l \Delta_l^\top) \odot (\sigma_w^2 A_{l-1}A_{l-1}^\top/\text{fan}_l + \sigma_b^2) + \sigma_w^2 A_L A_L^\top/n + \sigma_b^2,$$

at $O(N^2 \cdot \text{width})$ cost instead of $O(N^2 \cdot \text{params})$.

Kernel prediction after t gradient-descent steps. With step η and $g_0 = 0$, the prediction is exact for discrete steps:

$$g_t(X_*) = \Theta_{*X}\,\Theta_{XX}^{-1}(I - (I - \eta\Theta_{XX}/n)^t)\,y.$$

Two kernels go into it:

- the network's **own** Θ_{w_0}, giving the *linearised model*;
- Θ_∞, giving the *infinite-width prediction*.

What we measure and why

Tasks. Targets are ± 1 and the output is scalar, so every kernel is exact. Seeds 0–2 vary both the data draw and the initialisation.

- **Two moons**: ml-ladder B1's generator, noise 0.18, 256 training and 1,024 test points, standardised.
- **MNIST, even vs odd**: B2's normalisation, 1,000 training and 1,000 test images, 100 per digit.

Measurements.

- **Kernel at initialisation.** $\|\Theta_{w_0} - \Theta_\infty\|_F / \|\Theta_\infty\|_F$ on 256 inputs, for widths 32 to 8,192. Theory predicts a slope of $-\frac{1}{2}$.
- **Width sweep** ($\alpha = 1$; widths 64, 256, 1,024, 2,048). Full-batch gradient descent with $\eta = n/\lambda_{\max}(\Theta_\infty)$: 1,000 steps on moons, 500 on MNIST. At log-spaced steps, compare the test predictions with the linearised model and with the infinite-width prediction. Also record the **kernel drift** $\|\Theta_{w_T} - \Theta_{w_0}\|/\|\Theta_{w_0}\|$ on the training set.
- α **sweep** (width 512, α from 0.03 to 100), with the same quantities. The same η serves every α, because the initial dynamics do not depend on it.
- **A smooth control.** The same sweep with exact-erf GELU in place of ReLU. The linearised comparison needs only the empirical kernel, which autodiff gives for any activation.
- **Two controls added after reading the first results.**
  - The infinite-width kernel model trained to convergence, to tell "better features" from "further along".
  - Float32 against float64 at large α, to tell a precision floor from a real effect.

What I built

- **`src/jaxexp/ntk.py`**:
  - the analytic NTK and NNGP kernels;
  - the empirical NTK by the per-layer identity, for any activation;
  - the brute-force $JJ^\top$, used only to check it;
  - the exact discrete-time kernel-regression path.
- **`src/jaxexp/data.py`**: MNIST from Google's CVDF mirror, each file checked against a recorded SHA-256. Their MD5s are LeCun's originals.
- **`experiments/n7_ntk/run.py`**: the six parts and the figure, checkpointing every run in `runs/`.
- **`tests/test_n7_ntk.py`**, 4 tests:
  - the per-layer NTK equals $JJ^\top$;
  - the empirical NTK approaches the analytic one with width;
  - the analytic diagonal equals its closed form;
  - the kernel path starts at 0, takes one exact step, and interpolates in the limit.

Checks against independent code.

- The per-layer empirical NTK equals brute-force $JJ^\top$ to 2.8×10^{-7}.
- The analytic kernels equal `neural-tangents` 0.6.5's, to 1.2×10^{-8} (NTK) and 7×10^{-15} (NNGP), on the same 64 MNIST images in float64. `neural-tangents` no longer imports under JAX 0.11, so it ran in a separate environment with JAX 0.4.30.

Results

The kernel at initialisation (relative Frobenius distance from Θ_∞, median of seeds):

width	32	256	2,048	8,192	slope, widths 256–8,192
moons	0.378	0.088	0.039	0.024	-0.41
MNIST	0.453	0.134	0.053	0.028	**-0.48**

Width sweep at $\alpha = 1$, measured at the end of training (median and range):

task	width	net vs linearised	net vs infinite-width	kernel drift	test accuracy	∞-width NTK, same step
moons	64	0.123 (0.112–0.164)	0.085	0.301 (0.217–0.521)	0.970	0.970
moons	2,048	0.047 (0.045–0.049)	0.038	0.019 (0.018–0.020)	0.970	0.970
MNIST	64	0.201 (0.201–0.211)	0.314	0.192 (0.165–0.226)	0.903	0.934
MNIST	256	0.105 (0.101–0.113)	0.224	0.115	0.921	0.934

task	width	net vs linearised	net vs infinite-width	kernel drift	test accuracy	∞-width NTK, same step
MNIST	1,024	0.050 (0.050–0.050)	0.128	0.031	0.932	0.934
MNIST	2,048	**0.034** (0.031–0.035)	0.096	**0.024** (0.016–0.031)	**0.932**	0.934

How the width sweep scales:

- **On MNIST the network approaches its own linearisation as width$^{-0.51}$** (0.201 → 0.034 over 32× width). Its kernel drift falls as width$^{-0.60}$.
- **The infinite-width prediction is a worse description than the network's own kernel.** At width 2,048 the network is 9.6% from the infinite-width prediction but 3.4% from its own linearisation. The difference is the kernel's error at initialisation: the linearised and infinite-width predictions differ by 10.1% from each other.
- **Linearisation is a description for a window of time, not forever** (top middle of the figure). At every width the gap grows with training. On moons at width 1,024 the median goes from 0.004 at step 10 to 0.057 at step 1,000.
- **This explains the flat small-width end.** Moons' gap at the end of training barely changes from width 64 to 256 (0.123 → 0.114), even though at step 100 it halves (0.095 → 0.044).

The α sweep at width 512 (end of training; median of 3 seeds; log-log slopes fitted over α = 1–30):

	ReLU, moons	ReLU, MNIST	GELU, moons	GELU, MNIST
net vs linearised, $\alpha = 1$	0.074	0.081	0.018	0.040
net vs linearised, $\alpha = 30$	0.034	0.014	5.7×10^{-4}	1.3×10^{-3}
slope	**−0.23**	**−0.51**	**−1.02**	**−1.00**
kernel drift, α = 1 → 30	0.044 → 0.0071	0.079 → 0.0042	0.064 → 6.3×10^{-4}	0.090 → 7.9×10^{-4}
kernel drift slope	−0.51	−0.84	−1.36	−1.39
weight change, $\alpha = 1 \to 30$	$8.5 \times 10^{-3} \to 3.1 \times 10^{-4}$	$6.9 \times 10^{-3} \to 2.3 \times 10^{-4}$	$9.1 \times 10^{-3} \to 3.0 \times 10^{-4}$	$6.9 \times 10^{-3} \to 2.3 \times 10^{-4}$

The precision control (moons, width 512, seed 0, float32 against float64 weights):

	$\alpha = 30$, f32	$\alpha = 30$, f64	$\alpha = 100$, f32	$\alpha = 100$, f64
ReLU, net vs linearised	2.87×10^{-2}	3.23×10^{-2}	1.77×10^{-2}	1.73×10^{-2}
GELU, net vs linearised	5.7×10^{-4}	6.7×10^{-4}	2.4×10^{-3}	2.0×10^{-4}

Test accuracy and the convergence control:

- **MNIST.** At $\alpha = 0.03$ the networks reach **0.951** (ReLU) and **0.954** (GELU). With $\alpha \geq 1$ they reach 0.912–0.917 (ReLU) and 0.926–0.929 (GELU), and the infinite-width kernel at the same step reaches 0.934.
- **The kernel catches up when trained longer.** Trained to convergence, the infinite-width kernel model reaches **0.950** (0.950 already at 10× the steps, with train MSE 1.1×10^{-3}).
- **Moons.** Every arm sits at 0.964–0.974, and the converged kernel at 0.970.

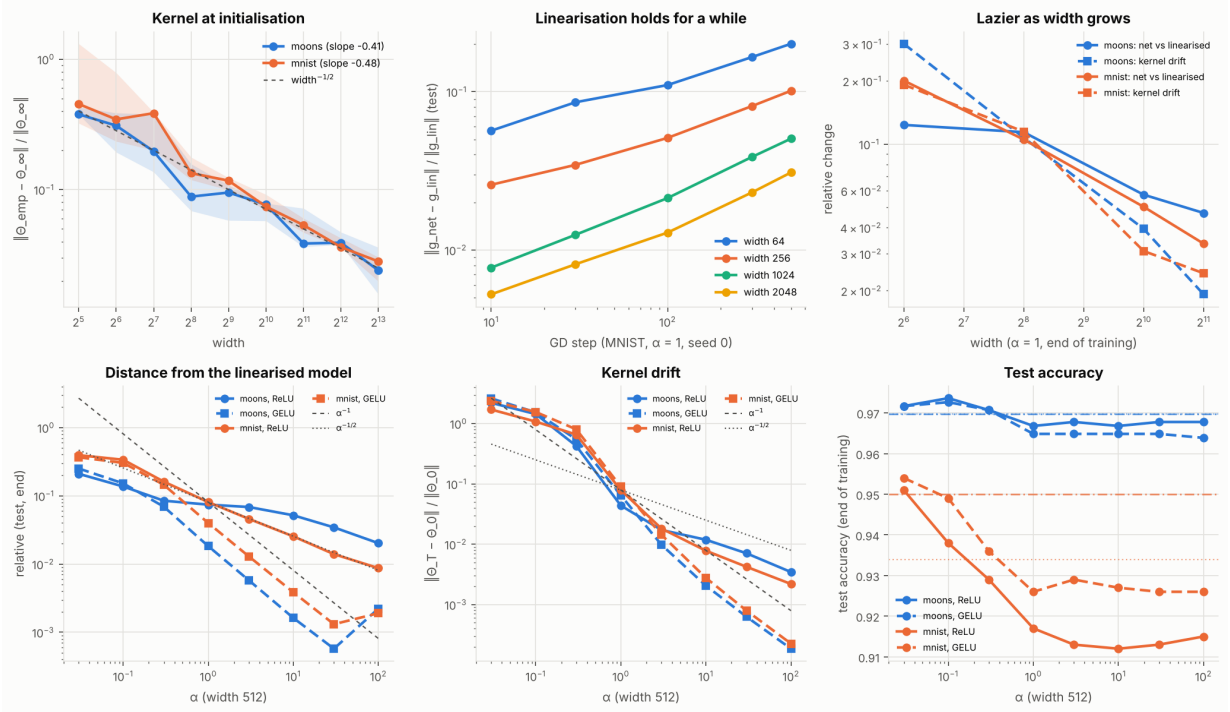


Figure 1: N7. Top: the empirical kernel at initialisation against width; the gap to the linearised model against training step for four widths; and the gap and kernel drift against width. Bottom: the gap and the drift against alpha for ReLU and GELU, with alpha^-1 and alpha^-1/2 guides; and test accuracy against alpha, where the dotted line is the infinite-width NTK at the same step and the dash-dot line the NTK trained to convergence.

Where the NTK picture holds, and where it doesn't:

- **"The kernel at initialisation converges as width$^{-1/2}$": holds.** The slope is -0.48 on MNIST and -0.41 on moons, the noisier of the two.
- **"Wide networks train as their kernel": holds, but for a window of time.** At width 2,048 the network is within 3.4% (MNIST) and 4.7% (moons) of its own linearisation. The gap grows with training at every width, so "lazy" means lazy for a number of steps that grows with width.
- **"The infinite-width kernel predicts the finite network": less well than its own kernel does.** At width 2,048 on MNIST the infinite-width prediction is 2.8× further away (9.6% against 3.4%).
- **"Lazy-training error falls as $1/\alpha$": holds for GELU, fails for ReLU.**
  - GELU gives slopes of -1.02 and -1.00.
  - ReLU gives -0.51 on MNIST and -0.23 to -0.35 on moons, with kernel drift near $\alpha^{-1/2}$.
  - This is not precision. Float64 weights give the same size of gap (1.73×10^{-2} against 1.77×10^{-2} at $\alpha = 100$). They are a different random draw, so the two are different networks, not the same network rounded differently.
- **"Feature learning beats the kernel": not shown here.** Small α moves the kernel by 170–

260% and gives the best accuracy at a fixed step count. But the kernel model trained to convergence matches it (0.950 against 0.951 and 0.954).

What surprised me

ReLU is not lazy the way the theorem says.

- Chizat and Bach's $1/\alpha$ bound assumes a Lipschitz Jacobian. A ReLU network's Jacobian jumps whenever a unit's pre-activation crosses zero.
- The weights move by $\propto 1/\alpha$ (measured: the weight change falls exactly as $1/\alpha$). That flips a fraction $\propto 1/\alpha$ of the units for some input.
- Each flip changes the kernel by $\pm O(1/n)$ in no particular direction, so the changes add up incoherently. The total grows as $\sqrt{\#\text{flips}} \propto \alpha^{-1/2}$.
- The smooth control removes the effect. Swapping ReLU for GELU, with nothing else changed, restores $1/\alpha$ to two digits of slope on both tasks.
- So the measured ReLU slopes, -0.5 and shallower, are the kinks' signature. The theorem's premise was doing real work.

The first explanation was wrong twice.

- **The flat ReLU curve:** my first guess was float32. A float64 rerun gave the same size of gap (1.73 against 1.77×10^{-2} at $\alpha = 100$).
- **GELU's upturn at $\alpha = 100$:** this one *is* float32. There, each step changes the weights by about 10^{-7} relative, float32's resolution. Float64 puts the point back on the line (2.0×10^{-4}, against 6.7×10^{-4} at $\alpha = 30$, a ratio of 3.3 for 3.3× in α).
- Same symptom, opposite causes, and only a rerun told them apart.

Feature learning looked like an accuracy win until the kernel got the same budget.

- At 500 steps, the small-α networks are 2–4 points ahead on MNIST.
- They have also driven train MSE to 0.001, against 0.135 for the lazy ones. Their kernel grows as the features move, so they train faster.
- Given 10× the steps, the infinite-width kernel reaches the same 0.950.
- On this task, at this scale, what feature learning bought was speed, not a better function. That is a statement about 1,000 MNIST digits and two hidden layers, not a general one.

The float32 arccos trap. On the kernel's diagonal $\cos\theta = 1$, where arccos has infinite slope. Float32 rounding of 10^{-7} in $\cos\theta$ becomes 3×10^{-4} in θ, which left the analytic NTK's diagonal 7×10^{-4} off. The comparison with `neural-tangents` caught it, and the kernels are now computed in float64.

Open threads

- **Measure the kink mechanism directly.** Count the activation flips per α and check that the kernel change is $\propto \sqrt{\text{flips}}$. Also check whether the ReLU exponent depends on width, as an incoherent sum over n units suggests.
- **A fair accuracy comparison.** Train every α to the same training loss, not the same step count, with η tuned per arm. That is the test of whether features help at all here.
- **Wider α at small width, and the NTK eigenspectrum.** Which kernel eigendirections the trained kernel grows into is the usual way to see what "learned features" means.
- **Depth.** Two hidden layers only. The NTK's deep limit degenerates, and how fast finite networks follow it is the next question.

O7 — Mel front-end parity

Not started. See `PLAN.md`.